# Introduction to Mutimodal AI for Biomedical Applications

[ W3-1: Neural Network – Architecture ]

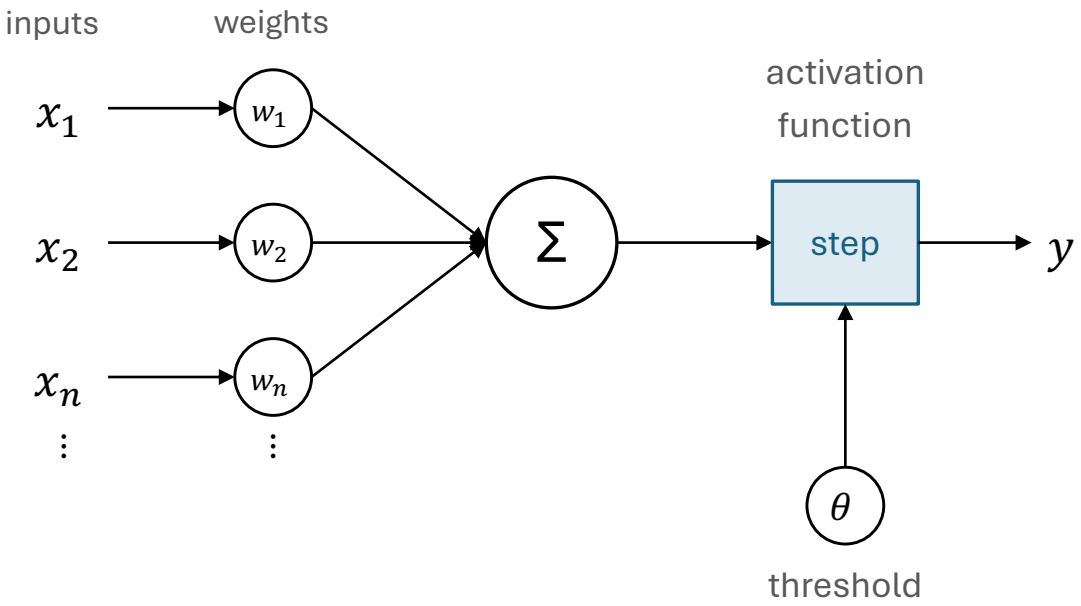
Jun-Hyuk Kim

Department of AI

# Overview

- One neuron is one straight line
- Neurons side by side: layers, depth, width
- Solving XOR by hand
- Why non-linearity is the missing piece
- Running the network: forward propagation
- How far can this structure go?

# The M-P Neuron




$$w_1 x_1 + \cdots + w_n x_n \geq \theta \rightarrow y = 1$$

$$\text{otherwise} \rightarrow y = 0$$

$x_i$: inputs, each 0 or 1

$w_i$: weights, how much each input counts

$\theta$: threshold, the level to reach

Every neuron ends with **an activation function.** Here, it is **the step function**.

# Nothing Here Is Learned

A neuron holds three kinds of numbers:

**Inputs**

$x$

given by the data

**Weights**

$w$

chosen by a person

**Threshold**

$\theta$

chosen by a person

There is no rule anywhere that improves $w$ or $\theta$ from experience.

The M-P neuron cannot learn. Remember this - we come back to it at the end.

# Building AND and OR

**AND**

$\boldsymbol{w} = (w_1, w_2) = (1, 1) \quad \theta = 2$

| $x_1$ | $x_2$ | Sum | $\geq 2$ ? | $y$ |
|---|---|---|---|---|
| 0 | 0 | 0 | ✗ | 0 |
| 0 | 1 | 1 | ✗ | 0 |
| 1 | 0 | 1 | ✗ | 0 |
| 1 | 1 | 2 | ✓ | 1 |

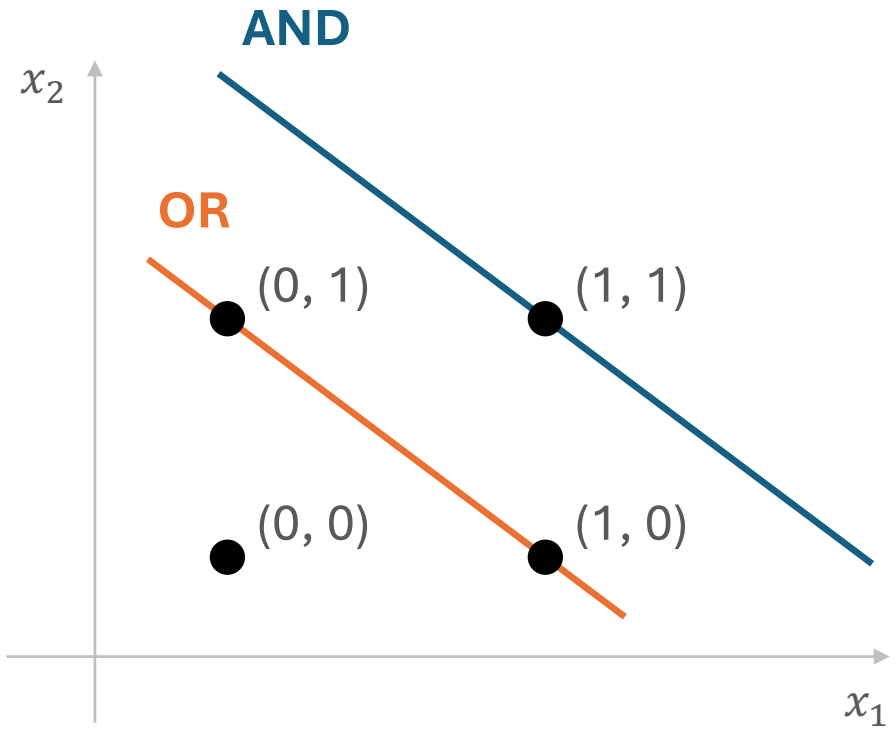
**OR**

$\boldsymbol{w} = (w_1, w_2) = (1, 1) \quad \theta = 1$

| $x_1$ | $x_2$ | Sum | $\geq 1$ ? | $y$ |
|---|---|---|---|---|
| 0 | 0 | 0 | ✗ | 0 |
| 0 | 1 | 1 | ✓ | 1 |
| 1 | 0 | 1 | ✓ | 1 |
| 1 | 1 | 2 | ✓ | 1 |

Same weights. Only $\theta$ changed - and the operation changed with it.

# That Boundary Is a Straight Line




AND boundary: $x_1 + x_2 = 2$

OR boundary: $x_1 + x_2 = 1$

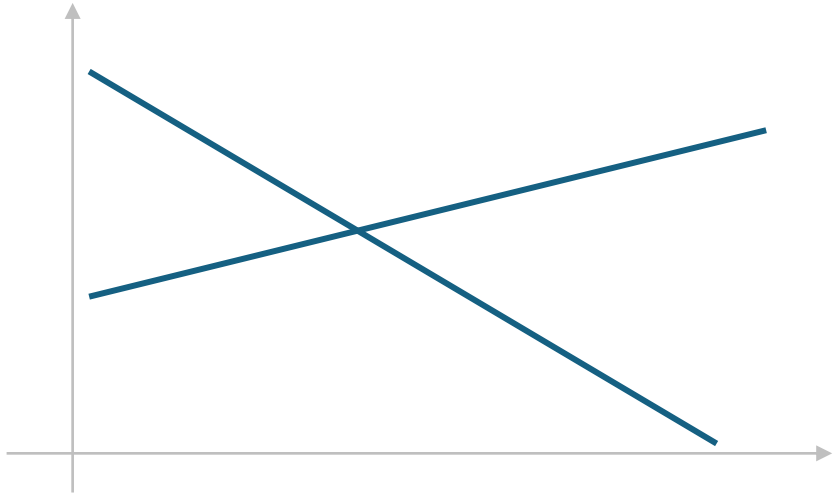
Both are ordinary straight lines.

**One neuron**
**= One straight cut through space**

Every network we build today is just a combination of lines.

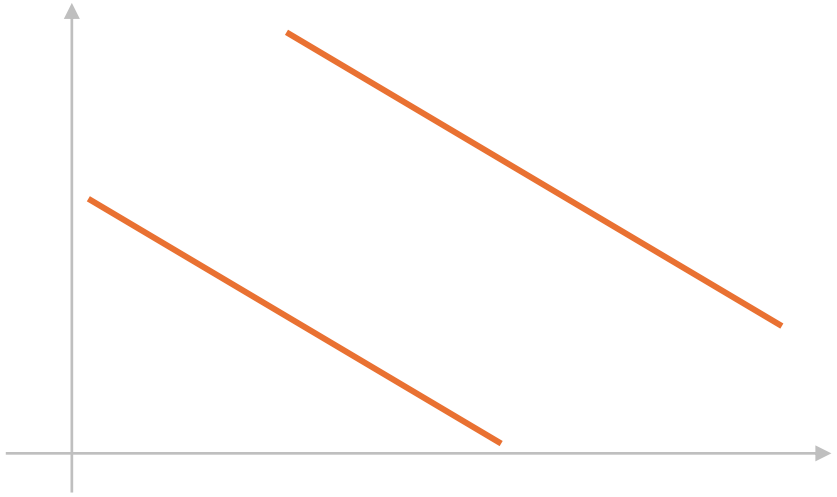
# Who Controls the Line?

$\boldsymbol{w}$ sets the direction



Change the weights and the line tilts.

$\theta$ sets the position



Change the threshold and the line slides.

The line comes from $w_1, w_2$ and $\theta$ together.

# From Threshold to Bias

$$w_1 x_1 + w_2 x_2 \geq \theta$$

write $b = -\theta$

$$w_1 x_1 + w_2 x_2 + b \geq 0$$

Same inequality. Nothing has been gained yet -

but bias $b$ now sits on the same side as the weights.

Next: why that small move matters.

# A Bias Is Just Another Weight

Add one imaginary input that is always 1:

$$w_0 x_0 \; + \; w_1 x_1 \; + \; w_2 x_2 \; \geq \; 0$$

with $x_0 = 1$ and $w_0 = b$

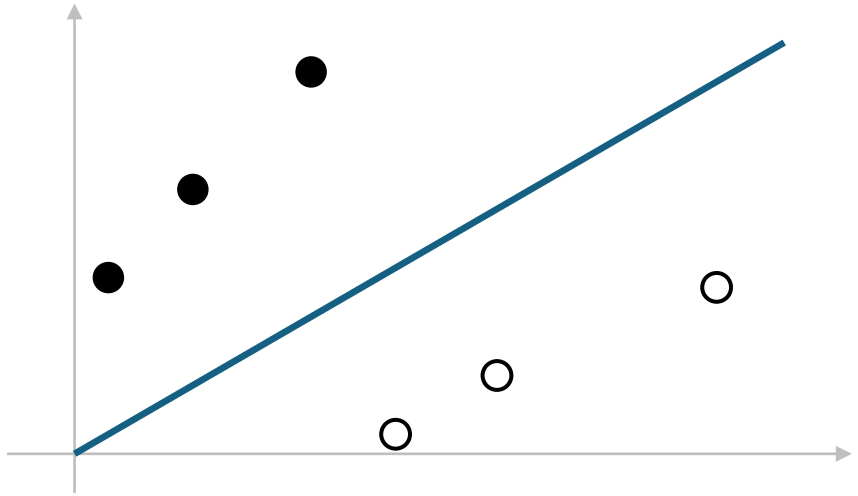
Rewriting $\theta$ did not make it learnable.

It made the bias **the same kind of object as a weight**,
thus, a single update rule can handle both.

From here on, every neuron carries a bias.

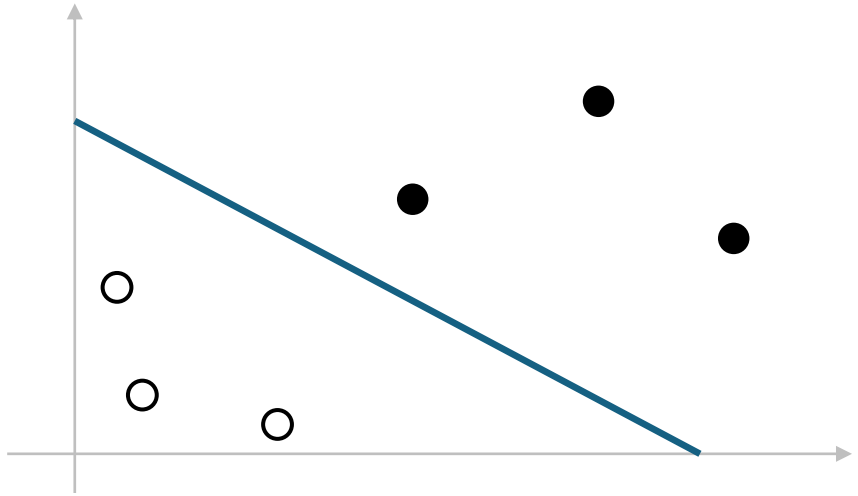
# What the Bias Does

$b = 0$



The line must pass through the origin

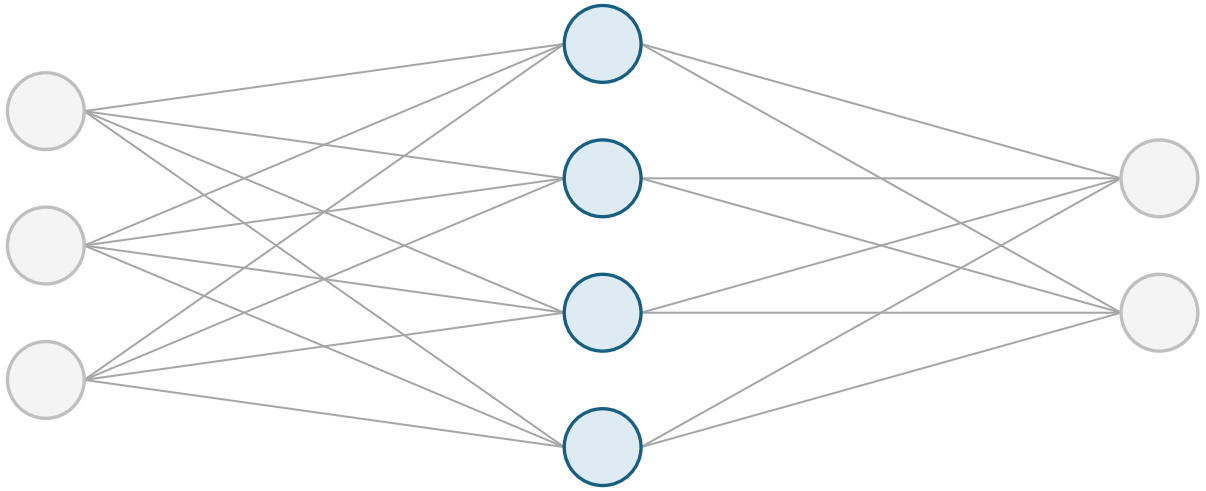
$b \neq 0$



The line can sit anywhere

Without a bias you cannot even build AND — its boundary $x_1 + x_2 = 2$ misses the origin.

# Neurons Side by Side: A Layer



**Input Layer**

holds the raw values

no computation

**Hidden Layer**

makes its own cuts

on what it receives

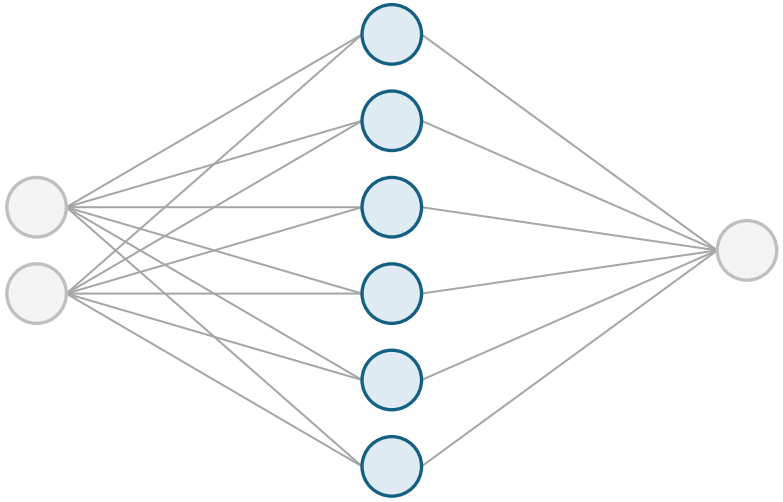
**Output Layer**

produces the answer

A layer of $m$ neurons draws $m$ cuts at once, independently.

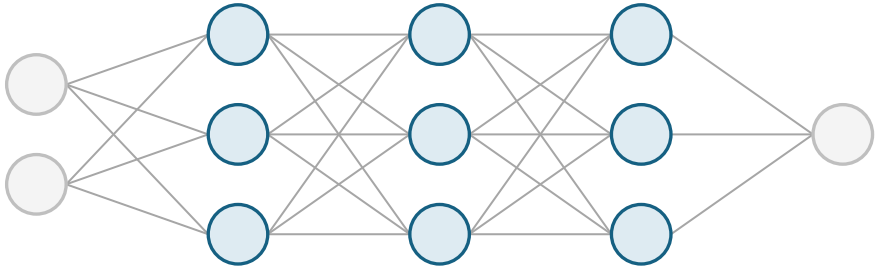
# Depth and Width

**Wide and shallow**



**Width:** how many cuts at once

**Narrow and deep**



**Depth:** how many times we repeat

Both add capacity. Both also add numbers that somebody has to choose.

# Counting the Numbers

A layer with $n$ inputs and $m$ neurons: $m \times n$ **weights** + $m$ **biases**

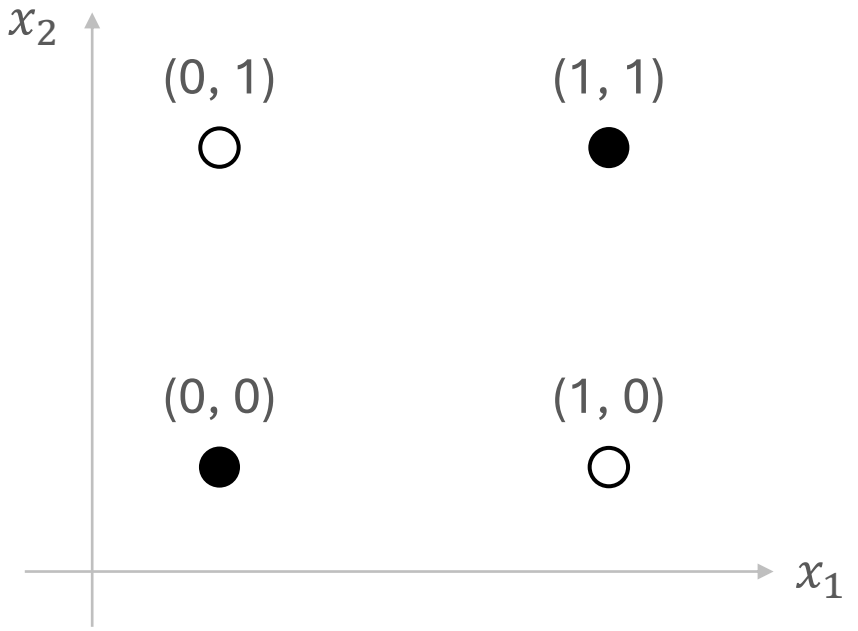
For the small network we are about to run:

| Layer | Weights | Biases |
|---|---|---|
| hidden (2 neurons) | 2 × 2 = 4 | 2 |
| output (1 neuron) | 1 × 2 = 2 | 1 |
| total | 6 | 3 |

Nine numbers. Keep that figure in mind.

# XOR: One Line Is Not Enough

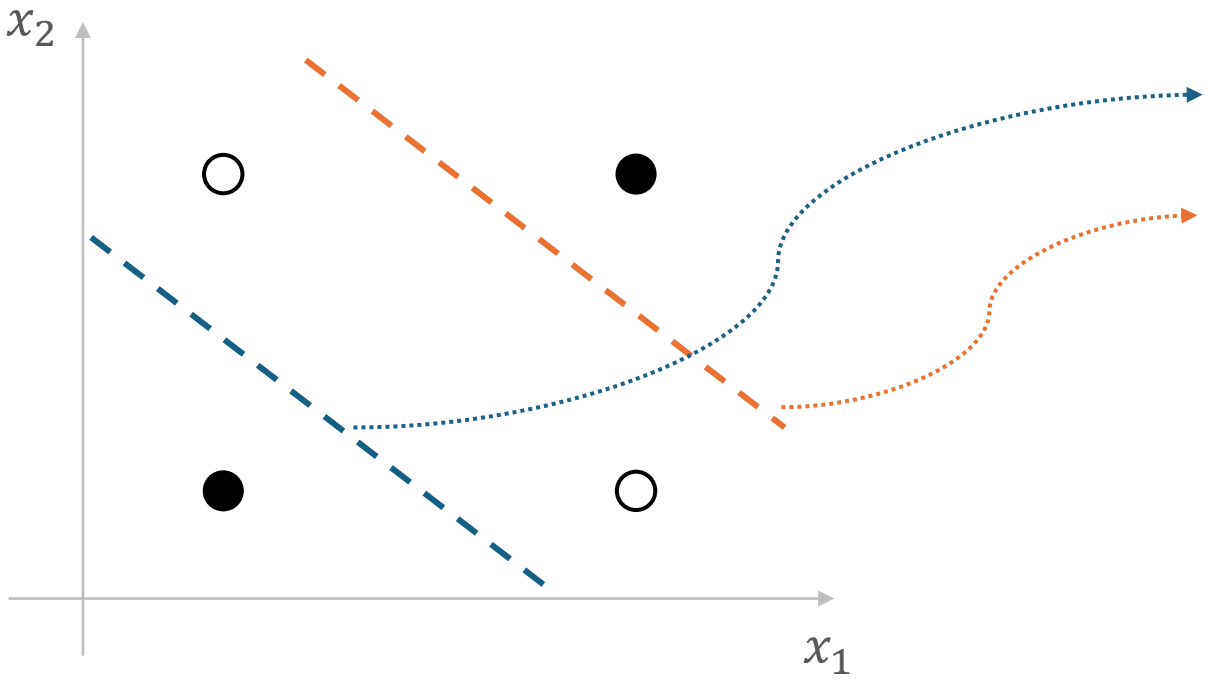
| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |




The white points sit on opposite corners.

No straight line separates them. **One neuron can never produce XOR.**

# XOR: Use Two Lines




**Line 1** $x_1 + x_2 \geq 0.5$ (OR)

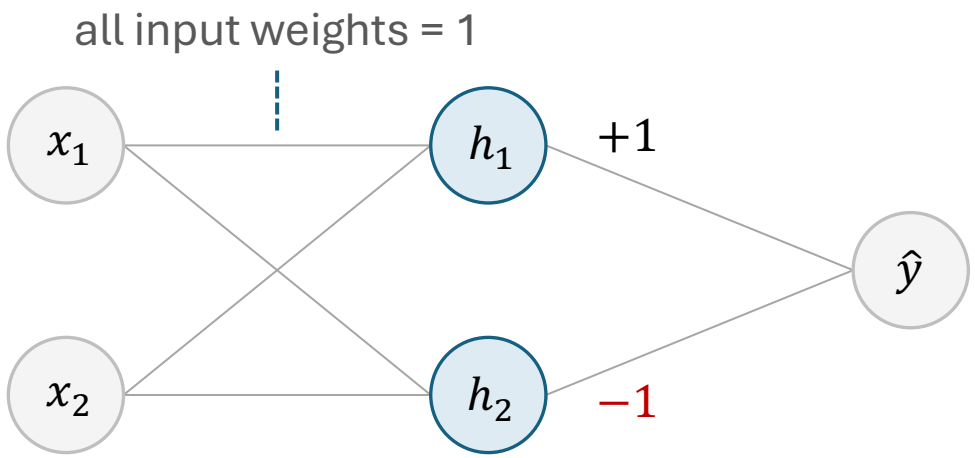
**Line 2** $x_1 + x_2 \geq 1.5$ (AND)

The white points lie

**between the two lines.**

XOR means: OR, but not AND.

# XOR: Build It




$h_1 = \text{step}(x_1 + x_2 - 0.5)$ : acts like **OR**

$h_2 = \text{step}(x_1 + x_2 - 1.5)$ : acts like **AND**

$\hat{y} = \text{step}(h_1 - h_2 - 0.5)$ : **OR** but not **AND**

Two hidden neurons draw one line each. The output neuron draws a third.

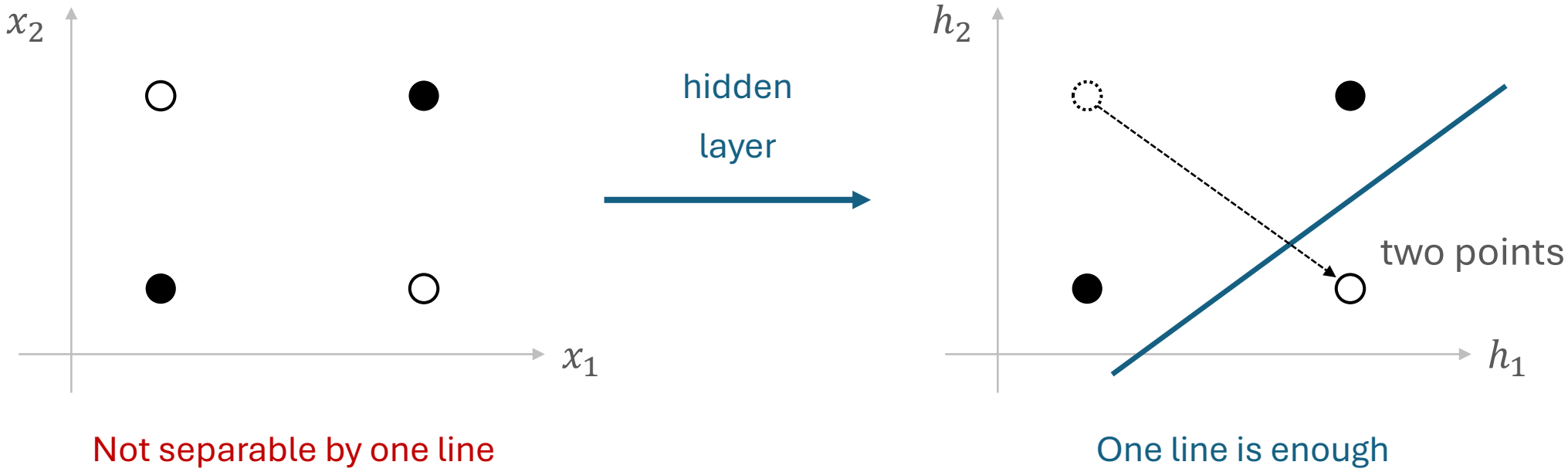
# XOR: Check Every Case

| $x_1$ | $x_2$ | $h_1$ | $h_2$ | $h_1 - h_2 - 0.5$ | $\hat{y}$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | −0.5 | 0 |
| 0 | 1 | 1 | 0 | +0.5 | 1 |
| 1 | 0 | 1 | 0 | +0.5 | 1 |
| 1 | 1 | 1 | 1 | −0.5 | 0 |

The last column is exactly the XOR truth table.

Not magic. Three straight lines, combined.

# The Hidden Layer Folds the Space




Not separable by one line

One line is enough

A hidden layer does not solve the problem. It redraws it as an easier one.

# Stacking Questions on Questions

### In our XOR network

$h_1$: "Is at least one of them 1?"

$h_2$: "Are both of them 1?"

We only supplied $x_1$ and $x_2$.

The layer invented these questions itself.

### In a face-recognition network

| Layer | The question it asks |
|---|---|
| input | pixel values |
| 1st | is there an edge here? |
| 2nd | do these edges form an eye? |
| 3rd | do the parts form a face? |

Each layer builds its question from the answers of the layer below.

# Layers Alone Are Not Enough

Suppose every layer is purely linear, with no activation:

$$
\begin{aligned}
a &= W_2(W_1 x + b_1) + b_2 \\
&= (W_2 W_1)x + (W_2 b_1 + b_2) \\
&= W' x + b'
\end{aligned}
$$

which is a single layer again

Stack a hundred linear layers and you still have one straight line.

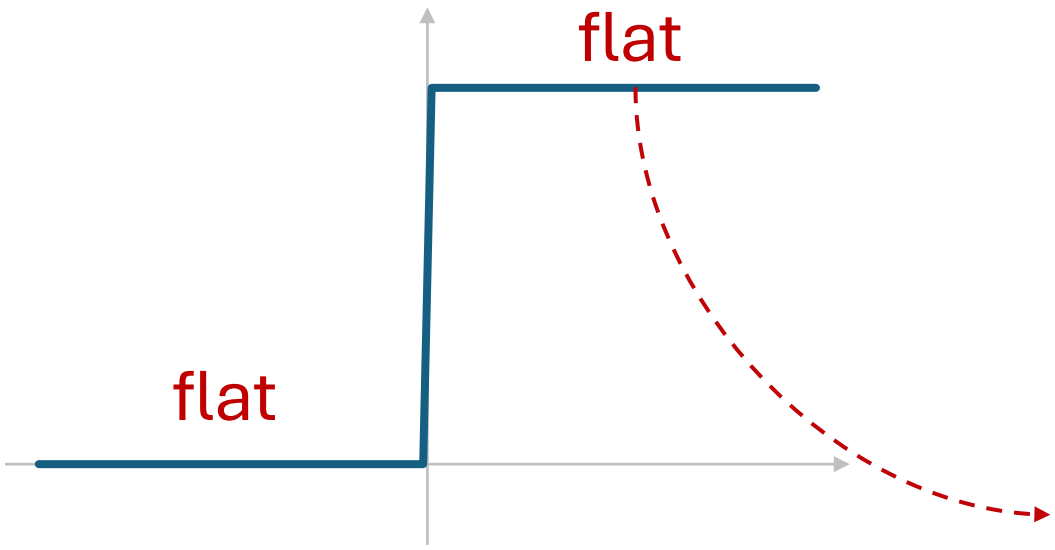
# Both Ingredients Are Needed

| What we have | What we can draw |
|---|---|
| many layers, no non-linearity | one line |
| non-linearity, only one neuron | one line |
| many layers and non-linearity | XOR solved |

Remove either ingredient and the network collapses back to a single line.

The activation function is what keeps the layers apart.

# Why the Step Function Has to Go




the step function

**How a machine would search**

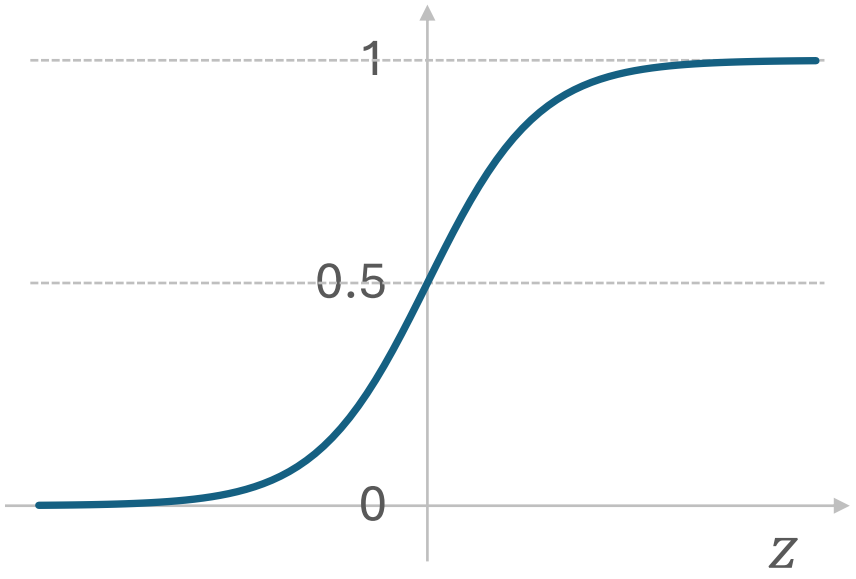
- start from any values
- make a prediction
- **change one value a little**
- keep the change if it helps

A small change gives **no change at all**.

Better or worse? There is no way to tell.

We need a smooth function: a small change in gives a small change out.

# Sigmoid

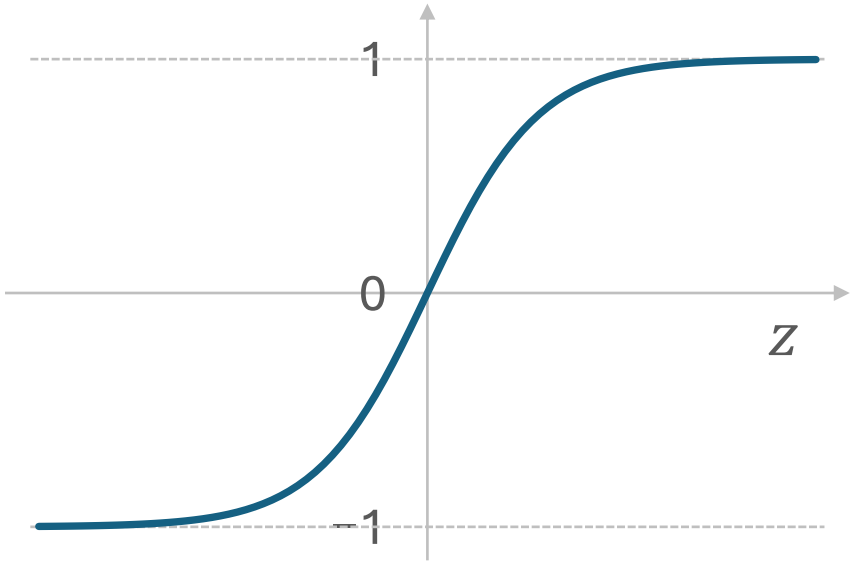



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Smooth everywhere,
  so a small change now does something
- Output stays between $0$ and $1$,
  so it reads as a probability
- Both ends flatten out again: **saturation**

It fixes the step function, but only near the middle.

# Hyperbolic Tangent (Tanh)

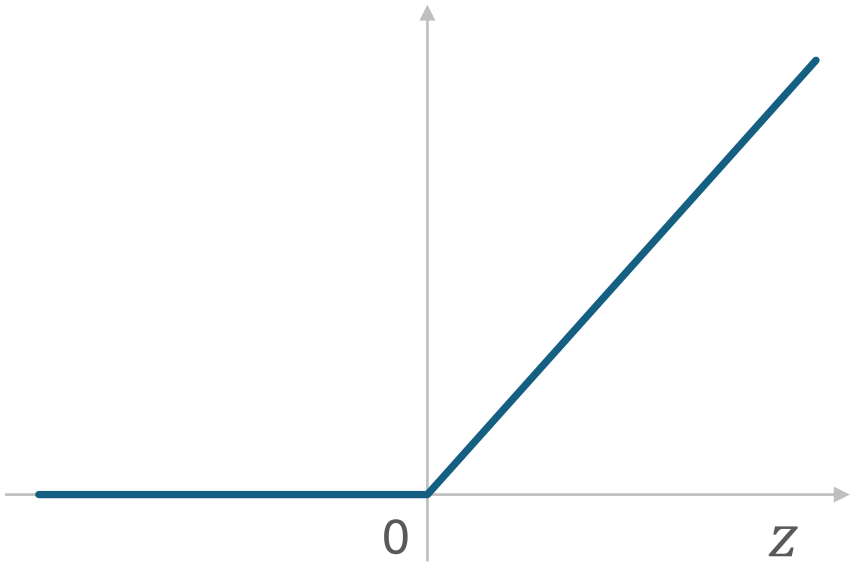



$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

- Same S-shape,
  but the output runs from $-1$ to $1$
- Centred on zero,
  so values do not drift to one side
- **Still saturates** at both ends

Better than sigmoid, but saturation is still there.

# Rectified Linear Unit (ReLU)

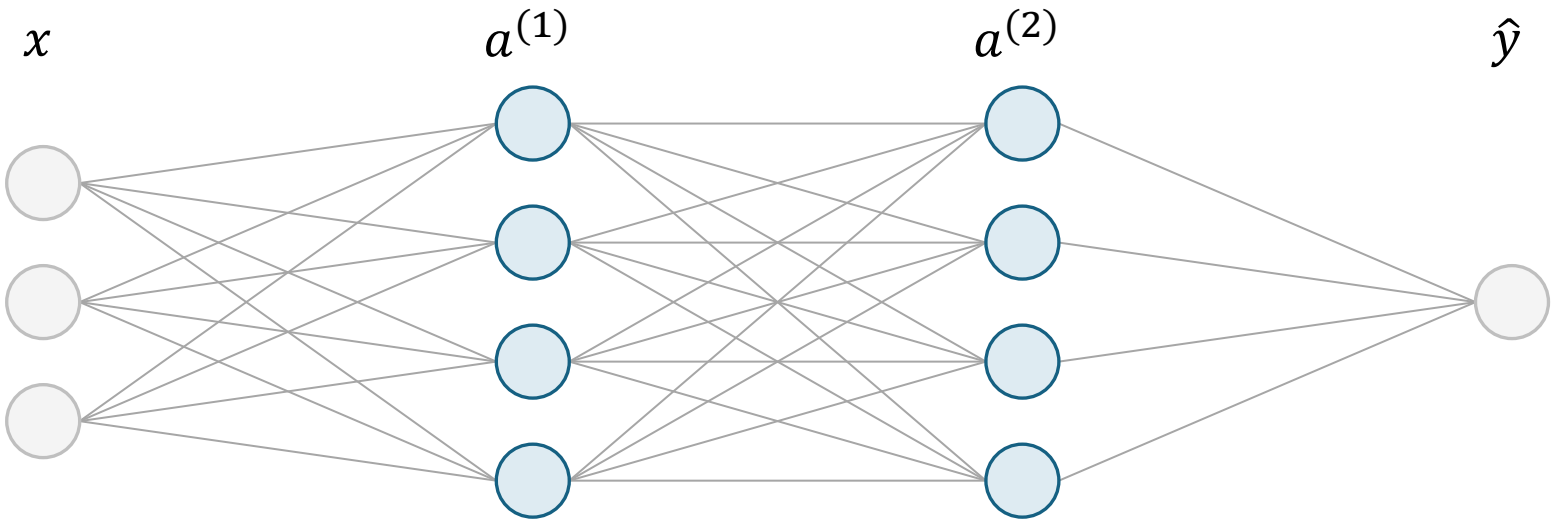



$$\mathrm{ReLU}(z) = \max(0, z)$$

- Negative goes to zero.
  Positive passes through unchanged.
- For every $z > 0$ the slope is exactly 1
  **no saturation**
- A neuron stuck below zero stops responding: **dying ReLU**

The first activation that really removed saturation.

# Forward Propagation




**Forward** →

Every layer does the same two steps: **weighted sum, then activation**

Its output becomes the next layer's input. No loops, no going back.

The superscript in $a^{(1)}$ is a layer number, not a power.

# Writing a Whole Layer at Once

Two neurons means two lines:

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

Arrange the numbers in a grid – a matrix:

$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} \qquad \boldsymbol{z = W\,x + b}$$

Nothing new is computed. Many lines are simply written as one.

# Reading the Matrix

$w_{ij}$: from input $j$ to neuron $i$

receiver first, sender second

The first row, $w_{11}x_1 + w_{12}x_2$, is exactly the first line from the previous slide.

So, **one row of $W$ is one neuron**. A layer of 100 neurons is a matrix with 100 rows.

Each row holds every weight belonging to one neuron.

# Worked Example: Input to Hidden

Given $x = \begin{bmatrix}1\\2\end{bmatrix}$ $W^{(1)} = \begin{bmatrix}1 & -1\\0.5 & 2\end{bmatrix}$ $b^{(1)} = \begin{bmatrix}0\\-1\end{bmatrix}$ $f = \mathrm{ReLU}$

**Step 1 weighted sum**

$$z_1 = (1)(1) + (-1)(2) + 0 = \mathbf{-1}$$

$$z_2 = (0.5)(1) + (2)(2) + (-1) = \mathbf{3.5}$$

**Step 2 activation**

$$a^{(1)} = \mathrm{ReLU}([-1,\ 3.5]) = \mathbf{[0,\ 3.5]}$$

The first neuron switched off. Different inputs wake up different neurons.

# Worked Example: Hidden to Output

Given $a^{(1)} = \begin{bmatrix} 0 \\ 3.5 \end{bmatrix}$ $W^{(2)} = [1 \quad 0.5]$ $b^{(2)} = [-1]$ $f = \text{sigmoid}$

**Step 1 weighted sum**

$$z^{(2)} = (1)(0) + (0.5)(3.5) + (-1) = \mathbf{0.75}$$

**Step 2 activation**

$$\hat{y} = \sigma(0.75) = \frac{1}{1 + e^{-0.75}} = \mathbf{0.68}$$

$x = [1, 2]$ → $z^{(1)} = [-1, 3.5]$ → $a^{(1)} = [0, 3.5]$ → $z^{(2)} = 0.75$ → $\hat{y} = 0.68$

A network is a function: numbers in, numbers out. Multiply, add, squash, repeat.

# How Far Can This Go?

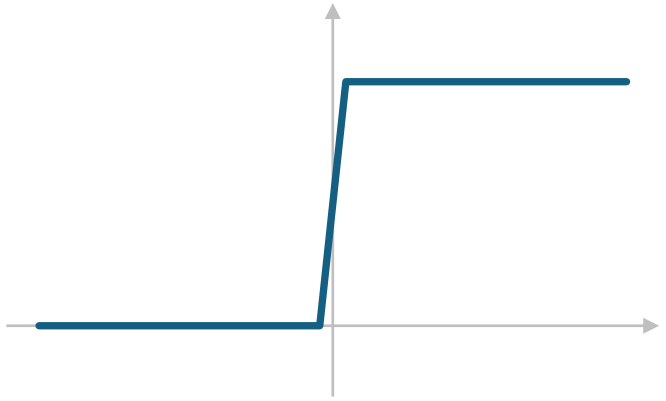
A network with **one hidden layer** and **enough neurons**
can copy any continuous function as closely as we want.

| The phrase | What it means |
|---|---|
| one hidden layer | you do not even need to go deep |
| enough neurons | no limit is placed on the number |
| continuous function | any unbroken curve, drawn without lifting the pen |
| as closely as we want | any accuracy you like, given more neurons |

This is the Universal Approximation Theorem (1989).

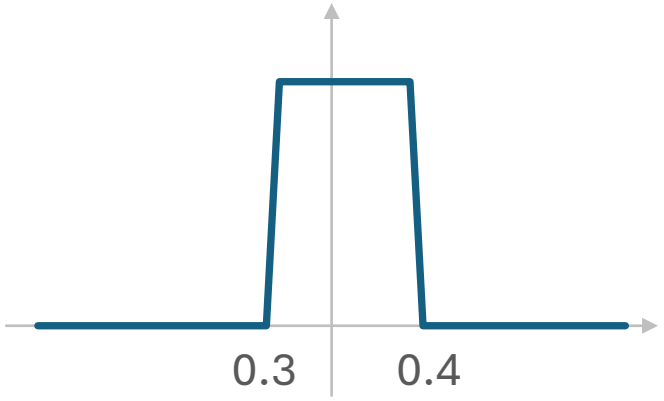
# Why It Works: Bars Under a Curve

**1. one neuron, one step**



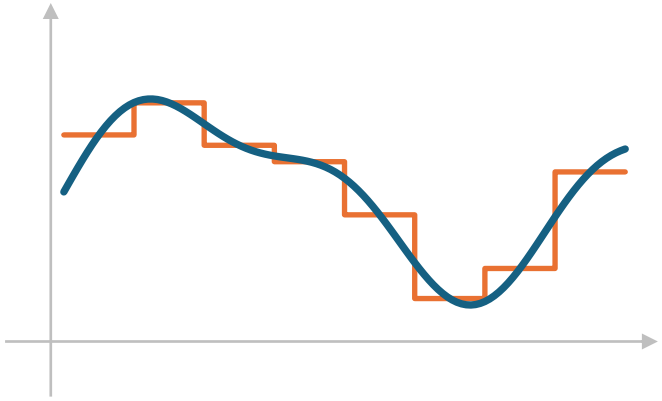
a large weight turns sigmoid into a step

**2. two steps, one bar**




subtract one step from the other

**3. many bars, any curve**



one bar per slice; thinner slices fit better

One bar costs two neurons – and every bar fits in the same hidden layer.

# What the Theorem Does Not Say

**It does not say how many neurons.**

One input needed 200 bars. Two inputs need a grid of 100 by 100. A 28 by 28 image has 784 inputs.

**It does not say how to find the weights.**

It proves that such a network exists. It gives no method for finding one.

**It is not quite what we want.**

We never know the true function. We only have data points, and matching those is not the same as being right on new data.

Being possible and being practical are two different things.

# So Why Go Deep?

Think of cutting paper snowflakes.

Fold the paper a few times, make **one cut**, and you get many holes at once.

| **Wide and shallow** | **Deep** |
| --- | --- |
| cut every hole separately | fold first, then cut once |

Folding is exactly what the hidden layer did to XOR.

Depth reuses work. Width repeats it.

# Who Chose Those Numbers?

XOR: $-0.5,\ -1.5,\ -1,\ -0.5$

Worked example: $1,\ -1,\ 0.5,\ 2,\ 0,\ -1,\ 1,\ 0.5,\ -1$

I chose every one of them by hand.

| Network | Numbers to choose |
|---|---|
| our 2 → 2 → 1 | 9 |
| a small handwriting recogniser | about 100,000 |
| a modern language model | billions |

Next week: how a network finds these numbers on its own.